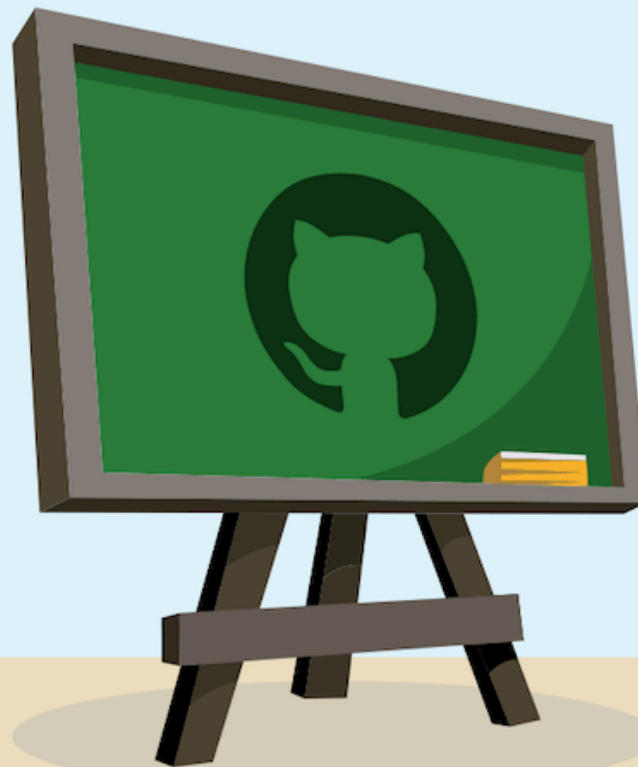




Gestão da Configuração e Versionamento

CÁSSIO CAPUCHO PEÇANHA



Iniciando...

Criar a Conta no GitHub (O Passaporte)

- Acessar github.com/join .
 - Um e-mail válido ou login com Google
- Criar um Username (Nome de Usuário).
- Dica de Ouro: Escolha um nome profissional!
 - Evitar coisas como `matador_de_dragao_99`. Esse perfil será o seu currículo no futuro.
- Verificar o e-mail clicando no link que o GitHub mandar. Sem isso, a conta fica travada.

Acessar o Github Classroom

Acesse o link:

<https://classroom.github.com/a/kt8clo5S>

09/03/2026



GitHub Classroom by **GitHub** would like permission to:

 Verify your GitHub identity (cassioaluno0)

 Know which resources you can access

 Act on your behalf
[? Learn more](#)

Resources on your account

 **Email addresses** (read)
View your email addresses

 **Plan** (read)
View your subscription plan on GitHub

[Learn more about GitHub Classroom](#)

Cancel

Authorize GitHub
Classroom

Authorizing will redirect to
<https://classroom.github.com>

Aceite o convite

Chegará também no e-mail o link para participar do projeto.

09/03/2026

GitHub



@github-classroom[bot] has invited you to collaborate on the
Turma-GCV-2026-1/tarefa-1-hello-world-cassioaluno0 repository

You can [accept](#) or [decline](#) this invitation. You can also head over to <https://github.com/Turma-GCV-2026-1/tarefa-1-hello-world-cassioaluno0> to check out the repository or visit [@github-classroom\[bot\]](#) to learn a bit more about them.

This invitation will expire in 7 days.

[View invitation](#)

Note: This invitation was intended for [cassioaluno0@gmail.com](#). If you were not expecting this invitation, you can ignore this email. If [@github-classroom\[bot\]](#) is sending you too many emails, you can [block them](#) or [report abuse](#).

Getting a 404 error? Make sure you're signed in as [cassioaluno0](#).

Button not working? Copy and paste this link into your browser:
<https://github.com/Turma-GCV-2026-1/tarefa-1-hello-world-cassioaluno0/invitations>

O que está acontecendo?

Quando você aceitou o convite que chegou no e-mail, o robô do GitHub fez o seguinte:

1. **Clonou (Copiou)** o meu repositório original (o modelo que eu criei).
2. **Criou um NOVO repositório** separado, com o nome do aluno (...-cassioaluno0).
3. **Deu permissão** para o aluno mexer nesse novo repositório.
4. **Deu permissão** para o professor ver o que ele está fazendo.

O que é um Fork? (A "Bifurcação")

- É uma cópia completa de um repositório que é salva na sua conta do GitHub.
- Para que serve?
 - Experimentação Segura: Você pode mexer no código à vontade sem medo de quebrar o projeto original.
 - Contribuição: É o primeiro passo para ajudar em projetos Open Source. Você copia (Fork), melhora no seu lado, e depois oferece a melhoria de volta (Pull Request).

O Comando `git clone`

- O que ele faz?
 - Baixa uma cópia completa do projeto.
 - Cria a pasta `.git` (o cérebro do Git) na sua máquina.
 - Conecta o seu computador ao servidor remoto (para enviar mudanças depois).
- Sintaxe:
 - `git clone` <https://github.com/usuario/projeto.git>
- A "Mágica" do Codespaces:
 - Quando vocês clicam no botão verde 'Create Codespace', o GitHub cria um computador virtual na hora e roda o comando `git clone` automaticamente para vocês. Por isso, quando a tela abre, os arquivos já estão lá esperando.

Abrindo o projeto.

- Code->Codespaces->Create codespace on main.
 - O projeto demora de 30 a 60 segundos para carregar.
- Abrir o arquivo main.
 - Realizar alguma alteração no arquivo.
- Executar:
 - `gcc main.c -o <nome programa>`
 - `./<nome programa>`

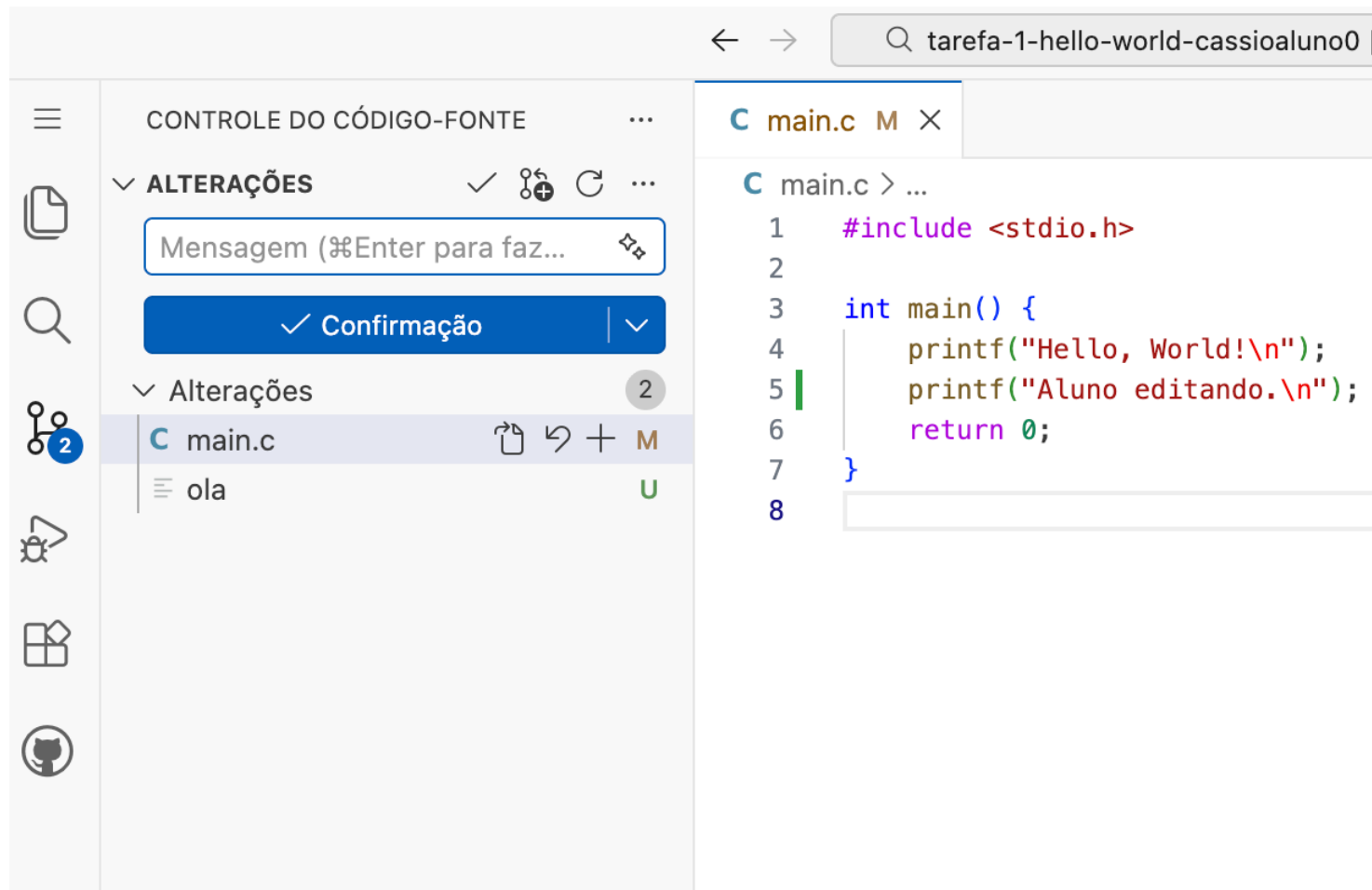
O Comando `git status`

- O que ele faz?
 - É o comando de diagnóstico.
 - Ele diz exatamente como o Git enxerga os seus arquivos naquele momento.
- Sintaxe:
 - `git status`
- **"Na dúvida, digite `git status`."**
 - Antes de commitar? Status. Deu erro? Status. Voltou do almoço? Status.

O Comando `git status`

- Changes not staged for commit
 - Mudanças não preparadas
 - O Git está dizendo: "Ei, eu já conheço esse arquivo main.c (ele já foi salvo antes), mas você mexeu nele. A versão que está no seu computador agora é diferente da última que você salvou."
- Untracked files
 - Arquivos não rastreados
 - O Git está dizendo: "Esses arquivos são novos, eu nunca vi na vida. Eu não vou cuidar deles a menos que você mande."
- **Atenção, com a deleção!**

Analise suas mudanças



O arquivo `.gitignore`

`.gitignore`: O que NÃO devemos versionar?

- O que é: Um arquivo de texto simples (sem extensão) que vive na raiz do projeto.
- Funciona como uma "lista de restrições". Ele diz ao Git quais arquivos ou pastas devem ser intencionalmente ignorados e não rastreados.
- É a "Capa da Invisibilidade" para arquivos que não queremos compartilhar.

O arquivo `.gitignore`

➤ **Versionamos a Receita, não o Bolo:**

- O Git deve guardar o Código Fonte (.c), que é a receita.
- O Git NÃO deve guardar o Executável (.exe, a.out), que é o bolo pronto.

➤ **Motivos:**

- Lixo Binário: Executáveis são pesados e mudam a cada compilação, sujando o histórico.
- Configurações Pessoais: Ninguém quer baixar as configurações de cor do seu VS Code (.vscode/).
- Segurança: Evitar subir arquivos com senhas ou anotações privadas.

O arquivo `.gitignore`

O Que Ignorar? (Exemplos Práticos)

- Artefatos de Compilação (Linguagem C):
 - *.exe, *.o, a.out, main (o arquivo sem extensão).
- Arquivos de Sistema e IDE:
 - .vscode/ (Configurações do editor).
 - .DS_Store (Arquivos temporários do Mac).
 - Thumbs.db (Arquivos temporários do Windows).
- Arquivos Temporários ou Privados:
 - *.log (Logs de erro).
 - rascunho.txt ou segredos.txt.

O arquivo `.gitignore`

Como Escrever (Sintaxe Básica)

➤ `#`

- Linhas começando com hashtag são comentários.

➤ `nome_do_arquivo`

- Ignora um arquivo específico.

➤ `*.extensão`

- O asterisco é um "coringa" (ignora *todos* com aquela extensão).

➤ `pasta/`

- A barra no final ignora uma pasta inteira.

```
# Ignorar executáveis gerados pelo GCC
main
ola
*.exe
*.out

# Ignorar configurações do meu computador
.vscode/
.DS_Store

# Ignorar meus rascunhos
segredos.txt
```

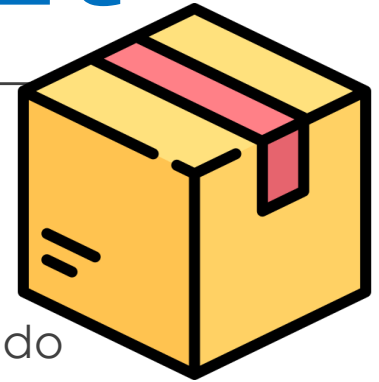

O Comando `git commit`

- **Não é "Salvar Arquivo", é "Salvar Momento".**
- Imagine que você está jogando um jogo difícil. Antes de entrar na sala do Chefão, o que você faz? Você cria um Save Point (Checkpoint).
- Se você morrer (o código quebrar), você volta exatamente para aquele ponto.
- Ctrl + S (Salvar no Editor): Sobrescreve o arquivo no seu HD. Você perdeu o que estava escrito antes.
- `git commit`: Tira uma FOTO (Snapshot) de como o projeto todo estava naquele momento e guarda no álbum histórico.
 - O arquivo antigo continua salvo na memória do Git.

O Comando **git commit**

O que tem dentro dessa "caixa"?

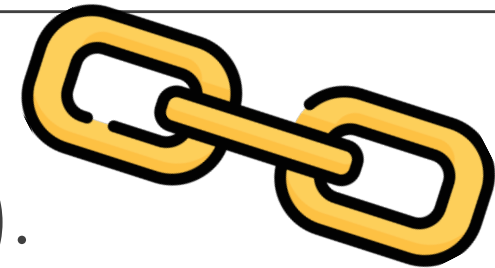
- Imagine uma caixa fechada e selada.
- **1. O Código (Snapshot):** Uma cópia exata de todos os arquivos do projeto naquele instante.
- **2. O Autor: Quem fez? (Nome + E-mail).** Importante para culpar o coleguinha depois.
- **3. A Data:** Quando foi feito?
- **4. A Mensagem:** Uma explicação humana do que mudou (Ex: "Corrigi o bug do loop infinito").
- **5. O Hash (O RG):** Um código único de 40 letras e números (ex: a1b2c3d...) que identifica aquele commit. Não existem dois iguais no universo.



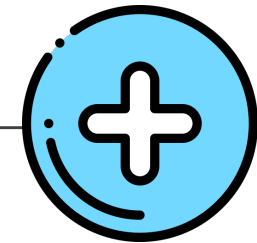
O Comando `git commit`

A História do Projeto

- Todo commit tem um PAI (Parent).
- O Commit B sabe que veio depois do Commit A.
- Isso cria uma corrente indestrutível. Se você tentar mudar algo no passado (Commit A), a corrente quebra (o Hash muda).
- O repositório do Git nada mais é do que uma fila imensa dessas "caixas" de commits, contando a história do seu software do início ao fim.



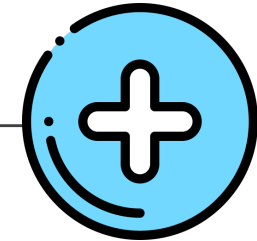
O Comando `git add`



Escolhendo quem sai na foto.

- Imagine que o Commit é tirar uma FOTO oficial da turma.
- O Diretório de Trabalho (Working Directory) é a bagunça do camarim: gente se trocando, gente comendo, gente pronta.
- O `git add` é o ato de chamar as pessoas para o PALCO.
- "Você (arquivo A), suba no palco. Você (arquivo B), espere aí que sua roupa está suja."
- O Fluxo:
 - Trabalho: Você edita 10 arquivos.
 - `git add. arquivo1.c`: Coloca só o arquivo 1 no palco.
 - `git commit`: Tira a foto APENAS de quem está no palco. Os outros 9 continuam lá no camarim (modificados, mas não salvos na história).

O Comando `git add`



O `git add` existe para dar Controle.

➤ Cenário Real:

- Você consertou um bug no `login.c`.
- Você também começou a fazer uma função nova no `menu.c` que está incompleta e quebrada.
- Se não tivesse `git add`: Você seria obrigado a salvar o bug consertado junto com o código quebrado.

➤ Com `git add`:

- `git add login.c` (Sobe só o conserto).
- `git commit -m "Arrumei o login"` (Salva o conserto).
- O `menu.c` continua lá, sujo, esperando você terminar.

O Comando `git pull`



Trazendo o mundo para casa

- Sincronizando sua máquina com o servidor.
- Sabe quando você vai abrir um jogo online e aparece "Update Required"?
 - O jogo precisa baixar os mapas novos que os desenvolvedores criaram enquanto você dormia.
- O `git pull` é exatamente isso: Você está dizendo ao Git "Vá lá no GitHub, veja o que mudou desde a última vez que eu olhei, e traga tudo para o meu computador agora."

O Comando `git pull`



A Mágica em Duas Etapas

- Conceito: O `git pull` na verdade é um atalho para dois comandos que rodam juntos.
- **`git fetch` (Buscar):** O Git vai na internet e baixa as caixas novas, mas não abre. Ele deixa no cantinho ("Olha, tem novidade, mas não misturei com seu código ainda").
- **`git merge` (Mesclar):** O Git pega essas caixas e mistura com o seu código atual. Se tudo der certo, o código novo aparece magicamente no seu editor.

O Comando **git push**



➤ O momento da verdade

- Enviando suas alterações para a nuvem.
- git commit é tirar uma selfie e aplicar filtros (está salvo no seu celular, mas ninguém viu).
- git push é clicar em "Publicar" no Instagram.
 - Agora a foto está na rede, seus amigos podem ver, curtir e comentar.

O Comando `git push`



➤ Por que o `git push` falha?

- A Regra do "Primeiro Baixa, Depois Sobe".
- O Problema (Rejected):
 - O Git é muito educado (e teimoso). Ele nunca vai deixar você "atropelar" o trabalho de outra pessoa.
 - Se alguém (ou o professor) mandou algo para o servidor enquanto você trabalhava, o servidor mudou.

O Comando `git push`



- Imagine que o repositório no GitHub é um Trem que só anda para frente.
 - Cada vagão é um Commit.
- A Regra Sagrada do Git: "Você só pode engatar seu vagão no ÚLTIMO vagão do trem."
- Você olhou para o trem e o último vagão era o Vagão 10.
- Você foi para a sua oficina (local) e construiu o Vagão 11, feito sob medida para engatar no Vagão 10.
- Enquanto você construía, seu colega foi lá e engatou o vagão dele. Agora o trem termina no Vagão 11 (do colega).
- Você chega correndo com o seu vagão e tenta engatar ele no Vagão 10 (tentativa de push).
- O Git (Maquinista) grita: "Ei! O Vagão 10 já tem um engate! O último agora é o 11."
 - Isso é o erro Rejected (Non-fast-forward).